Intune Wi-Fi Profile Remediation — Changes Not Applying (Proxy, SSID & Meraki)

Implementation Solution Guide

Version 1.0.0 · 2026-08-05

Contents

Executive Summary 4

1. Solution Overview — Intune Wi-Fi Profile Remediation (Changes Not Applying) 4

The problem this solves . . . 4

Why a modified Wi-Fi profile so often does not update in place — the crux . . . 5

How a Wi-Fi change travels — and where it stalls . . . 5

The Meraki interaction — which system is authoritative . . . 6

What you get . . . 6

The repeatable method . . . 7

2. Requirements Guide — Intune Wi-Fi Profile Remediation 7

2.1 Intune administrator role (or Policy & Profile Manager) . . . 7

2.2 Microsoft Graph read access (for the scripts) . . . 7

2.3 Graph scopes for the remediation helper . . . 7

2.4 Access to at least one affected device (per platform) . . . 8

2.5 Certificate dependency present (enterprise/802.1X profiles only) . . . 8

2.6 A reproducing device and a known-good device . . . 8

2.7 Clarity on whether Cisco Meraki is in scope . . . 8

Checklist . . . 9

Global Requirements — Intune Solutions (category-level) 9

G1. Microsoft Entra ID tenant . . . 9

G2. Microsoft Intune subscription . . . 9

G3. Administrative roles . . . 9

G4. Device enrollment in Intune . . . 10

G5. Pilot security group . . . 10

G6. Supported Windows baseline . . . 10

G7. Network access to Microsoft cloud endpoints . . . 10

3. Diagnostic Playbook — Wi-Fi Changes Not Applying 10

Step 0 — Establish the facts (always do this first) . . . 11

Branch A — Modified profile does not update the existing SSID in place . . . 11

Branch B — Proxy set wrong, in the wrong field, or as URL-vs-hostname . . . 13

Branch C — Duplicate or conflicting Wi-Fi profiles . . . 14

Branch D — Cisco Meraki is overriding the client settings . . . 14

Branch E — Profile reports Succeeded, device is unchanged (delivery vs application) . . . 15

Branch F — Platform-specific quirks (Windows vs iOS vs Android) . . . 16

After any fix — re-validate that it applied on a device . . . 17

5. Testing & Validation 17

Gate 0 — Reproduce and baseline (before any fix) . . . 17
Gate 1 — Corrected replacement profile exists and is scoped (Branch A) . . . 17
Gate 2 — The change is APPLIED on a previously-joined device (the core gate) . . . 18
Gate 3 — Proxy is in the correct shape and effective (Branch B) . . . 18
Gate 4 — Exactly one Wi-Fi authority for the SSID (Branches C & D) . . . 18
Gate 5 — Reported vs applied reconciled (Branch E) . . . 19
Gate 6 — New-device regression . . . 19
Promotion gate — Pilot → Production . . . 19
6. Troubleshooting 19
Connecting to Microsoft Graph . . . 20
Reading applied state on the device . . . 20
Interpreting states . . . 21
Remediation helper . . . 21
When the method stalls . . . 22
7. Best Practices 22
Treat a Wi-Fi change as a new profile, not an edit . . . 22
One profile — and one authority — per SSID per device . . . 22
Get the proxy shape right, every time . . . 22
Name objects so they audit and clean up cleanly . . . 23
Scope Wi-Fi profiles to devices that can use them . . . 23
Keep a fallback so a change never strands users . . . 23
Validate on the device, not in the console . . . 23
Keep vendor-specific and certificate steps in their lane . . . 23
Government / sovereign-cloud note . . . 23
8. Deliverables Summary 24
Documents . . . 24
PowerShell scripts (read-only by default; cloud-aware) . . . 24
Boundaries (what this does not cover — by design) . . . 25
Delivery . . . 25
Appendix A — PowerShell Scripts 26

Executive Summary

You changed a Wi-Fi setting in Intune — a new password, a proxy, a security type — the profile shows **Succeeded**, and the devices are still on the old configuration. Nothing you re-push, sync, or restart changes it. This is one of the most common and most confusing Intune symptoms, because every signal in the console says the change worked.

It usually did not fail. It **never reached the saved network on the device** — because Intune does not reliably rewrite a Wi-Fi network the device has *already* joined. Microsoft documents this directly: after you change the password on an existing Wi-Fi profile, *"some users may not get the new profile."* The same behavior applies to a changed proxy, SSID, or security type. The device keeps the network it already has, the profile still reports Succeeded (delivery, not application), and the admin is left re-pushing blindly.

This solution replaces that guesswork with a **symptom-driven remediation playbook** for Intune Wi-Fi configuration profiles that will not update across **Windows, iOS/iPadOS, and Android**. You start from what you observe, follow the branch, run the exact checks, and land on the one cause that owns the failure — then apply the reliable remediation: a **new, correctly-named replacement profile** and a controlled remove-and-re-add, rather than editing the stale profile in place and hoping.

It covers the two hardest real-world variants head-on:

- **Proxy placement.** Where the proxy actually lives in a Wi-Fi profile per platform, and the single

most common mistake — putting a URL or a `host/path` in the **manual** proxy field (which is a **host + port**, never a URL), or a bare hostname in the **automatic** field (which is a **PAC URL**). Get this wrong and the network connects but all traffic breaks.

- **Cisco Meraki.** How to tell whether the change is being masked by Meraki — either the **Meraki

dashboard **(the wireless-infrastructure side: APs, SSIDs, RADIUS, group policies) or** Meraki Systems Manager** (Meraki's own MDM, which pushes its *own* Wi-Fi payload and can compete with Intune for the same SSID) — and how to establish which system is authoritative before you touch the Intune profile.

You receive the diagnostic playbook, a requirements guide, testing gates that **prove the corrected settings actually applied on a device** (not just that Intune says so), a troubleshooting guide, best practices, and **five PowerShell scripts** (read-only by default) that export your Wi-Fi profiles and assignments, detect duplicates and conflicts, compare *reported* state against *applied* state on the device, and help you build the corrected replacement profile safely.

The result: the exact reason your Wi-Fi change is not reflecting — isolated to one cause — and a repeatable, low-risk pattern to fix it and keep it fixed, on every platform, on every future change.

1. Solution Overview — Intune Wi-Fi Profile Remediation (Changes Not Applying)

The problem this solves

You edit an Intune **Wi-Fi configuration profile** — change the pre-shared key, add or change a **proxy**, switch the security type, correct the SSID — save it, and it reports **Succeeded**. But the devices never change. They keep connecting with the old settings, the old proxy, or the old saved network. You re-push,

force a sync, reboot, and re-check: still Succeeded, still unchanged.

This toolkit replaces that loop with a **structured, symptom-driven method** specific to Wi-Fi profiles. You follow the change along the path it actually travels — from the Intune profile to the **saved network on the device** — isolate the one place it breaks, and apply the reliable remediation for that cause.

> ***Scope boundary.*** *This product owns **Wi-Fi configuration profiles that will not update or reflect a change**. It is *not* the general configuration-profile toolkit and *not* a Wi-Fi *deployment* product. For general "policy assigned but not applying" diagnosis across any configuration profile (assignment, filters, sync, CSP errors, conflicts), use the **Intune Device Configuration & Policy Troubleshooting Toolkit** — this Wi-Fi product reuses that layered model and points to it rather than repeating it. For first-time **802.1X / certificate-based Wi-Fi deployment** (SCEP/PKCS, trusted root, RADIUS), use the certificate-Wi-Fi and Wi-Fi-deployment products; this toolkit references them and does not reproduce certificate deployment.*

Why a modified Wi-Fi profile so often does not update in place — the crux

A device that has already joined a Wi-Fi network keeps that network as a **saved network** (a "known network" on Windows, a saved network on Android, an installed Wi-Fi payload inside the MDM profile on iOS/iPadOS). When you change the Intune profile, Intune re-delivers the payload — but the device does **not** reliably overwrite a network it already has with the same SSID.

Microsoft states this plainly for the most common case, a changed password: after you change the password on an existing profile and save it, *"some users may not get the new profile."* The same behavior is observed for other in-place edits (proxy, security type, auto-connect). The profile still reports **Succeeded** in Intune because **delivery succeeded** — the payload reached the device — even though **application** (rewriting the live saved network) did not. That split between *reported* and *applied* is the heart of nearly every "Wi-Fi change not reflecting" case.

Because of this, the **reliable remediation is not to edit the existing profile in place**. It is to deploy a **new, correctly-configured replacement profile** (a distinct profile object, and — where the change is to the network itself — often a controlled remove-and-re-add of the old saved network), so the device treats it as a new network to install rather than a known network to leave alone. This toolkit's playbook is built around that pattern.

How a Wi-Fi change travels — and where it stalls

A Wi-Fi change must pass through the same delivery layers as any configuration profile **and** one extra Wi-Fi-specific layer:

1. **Assignment & scope** — the profile is assigned to the right Entra group, in the right

(device vs user) context, not blocked by a filter. *(General layer — deferred to the configuration toolkit.)*

1. **Sync / delivery** — the device checks in and receives the payload. *(General layer.)*

2. **Certificate dependency** (enterprise/802.1X profiles only) — the Trusted Root and SCEP/PKCS

profiles the Wi-Fi profile depends on must be present first, or the Wi-Fi payload is skipped as *pending certificates*. *(Deferred to the certificate-Wi-Fi product; flagged here as a checkpoint.)*

1. **Application to the saved network** — **the Wi-Fi-specific layer this product owns.** Even after

the payload is delivered, the device must actually create or overwrite the saved network. For a *changed* profile targeting an SSID the device already knows, this is exactly where it silently does nothing.

1. **Report** — the device reports **Succeeded/Failed**, which reflects **delivery**, not whether the

live network on the device matches your intent.

The **Diagnostic Playbook** (document 3) starts at Step 0 (establish the facts) and then branches by symptom to the layer that owns each failure — with the in-place-update crux, proxy placement, and Meraki as the worked branches.

The Meraki interaction — which system is authoritative

When Cisco Meraki is in the environment, "changes not reflecting" often means **something other than Intune owns the setting**. Two very different Meraki products are commonly confused:

- **Meraki dashboard** — the **wireless-infrastructure** side: access points, broadcast SSIDs, RADIUS,

VLANs, and per-SSID **group policies**. This controls the *network*, not the device's saved-profile or client-side proxy. If the behavior you are chasing (e.g. a forced proxy, a splash page, an authentication method) is enforced by the AP/SSID, editing the Intune Wi-Fi profile will never change it — the dashboard is authoritative for that.

- **Meraki Systems Manager (SM)** — Meraki's **own MDM/UEM**, which pushes its own **Wi-Fi Settings

payload **(SSID, auto-join, and — on Apple only — proxy) to enrolled endpoints. If a device is enrolled in Meraki SM as well, it can carry a** second managed Wi-Fi profile for the same SSID**, and your Intune change may be masked by the Meraki payload. On** iOS/iPadOS a device can be MDM-enrolled to only one server**, so if it is enrolled in Meraki SM it is not taking Intune Wi-Fi profiles at all (and vice-versa) — a frequent root cause that no amount of Intune re-pushing fixes.

Branch D walks the exact checks to tell which system owns the setting. Meraki console internals change over time; where a Meraki-side step is version-sensitive, the playbook says **verify against current Cisco Meraki documentation** rather than guessing.

What you get

- A plain-language model of how a Wi-Fi change travels and why an in-place edit stalls (this document).
- A **Requirements Guide** (roles, Graph read access, device access) with verify/implement for each.
- A **Diagnostic Playbook** — Step 0 plus labelled branches A–F, each: symptom → how to confirm →

exact fix → re-validate.

- **Five PowerShell scripts**: preflight, export Wi-Fi profiles + assignments + duplicate/conflict

detection, on-device Windows validation (`netsh wlan` + effective proxy), reported-vs-applied state report, and a guarded remediation helper (`-WhatIf` by default) that builds the corrected/renamed profile or removes the stale saved network.

- **Testing** gates that prove the corrected settings actually applied on a device, plus

Troubleshooting and **Best Practices**.

The repeatable method

> ***Establish the facts → follow the change to the saved network → isolate the one cause → deploy a corrected replacement profile (don't edit in place) → prove it applied on a device.*** *The repeatable shape behind every branch:* ***Assess → Configure → Pilot → Validate → Expand.***

2. Requirements Guide — Intune Wi-Fi Profile Remediation

Confirm **every** requirement before you diagnose. A missing prerequisite (for example, an absent certificate dependency, or no access to an affected device) is itself a common reason a Wi-Fi change "isn't applying." Each entry follows **Description · Why required · How to verify · How to implement if missing.** Category-wide prerequisites are in `global-requirements-intune-solutions.md`.

> ***Naming convention (used verbatim throughout this toolkit).*** *Create and reuse these exact names so the steps, scripts, testing, and troubleshooting all line up. The core remediation pattern is a* ***new replacement profile*** *— never an in-place edit — so the names below are for the corrected objects you will build: - Pilot device group (Assigned):* `WIFIDX - Pilot Devices` *- Corrected Windows profile:* `WIFIDX - WiFi - Corrected - Windows - Pilot` *- Corrected iOS/iPadOS profile:* `WIFIDX - WiFi - Corrected - iOS - Pilot` *- Corrected Android profile:* `WIFIDX - WiFi - Corrected - Android - Pilot` *- Production rename (after pilot passes): the same names with* `- Prod` *in place of* `- Pilot` *Worked example values used in the playbook (replace with your own): SSID* `CORP-WIFI`*; manual proxy host* `proxy.corp.example` *port* `8080`*; PAC URL* `http://wpad.corp.example/proxy.pac`*.*

2.1 Intune administrator role (or Policy & Profile Manager)

Description: An Entra/Intune role able to **read and create** device configuration profiles and their assignments — **Intune Administrator**, or the built-in **Policy and Profile Manager** role. **Why required:** The remediation creates a **new** Wi-Fi profile and assigns it; read-only roles can diagnose but not remediate. **How to verify:** `https://entra.microsoft.com` → **Roles and administrators** → confirm your account holds **Intune Administrator** (or **Policy and Profile Manager**). **How to implement if missing:** Entra → **Roles and administrators** → assign **Intune Administrator**. Reserve Global Administrator for tenant-wide changes only.

2.2 Microsoft Graph read access (for the scripts)

Description: Delegated Microsoft Graph read scopes so the discovery/validation/report scripts can enumerate Wi-Fi configuration profiles, their assignments, and per-device state. **Why required:** The read-only scripts call Graph to export every Wi-Fi profile and detect duplicates and conflicts — the picture the console only shows one profile at a time. Without these scopes the scripts cannot sign in. **How to verify:** Run `WIFIDX-Preflight.ps1` — on first run it prompts for consent to `DeviceManagementConfiguration.Read.All`. Successful sign-in confirms access. **How to implement if missing:** Sign in as an admin who can consent, or have a Global Administrator grant admin consent for `DeviceManagementConfiguration.Read.All`, `DeviceManagementManagedDevices.Read.All`, and `Group.Read.All`. Install the SDK with `Install-Module Microsoft.Graph -Scope CurrentUser`.

2.3 Graph scopes for the remediation helper

Description: Write scope needed **only** when `WIFIDX-Remediation.ps1` creates the corrected replacement profile or pilot group for you (`DeviceManagementConfiguration.ReadWrite.All`, `Group.ReadWrite.All`). **Why required:** Creating a profile or group writes to Intune/Entra; the diagnostic scripts never need this. **How to verify:** The helper previews all actions under `-WhatIf` (its default) without needing write consent; the write scope is requested only when you re-run with `-WhatIf:$false`. **How to implement if missing:** Have a Global Administrator grant admin consent for the ReadWrite scopes before running the helper without `-WhatIf`. You can always build the profile by hand in the console instead (document 3).

2.4 Access to at least one affected device (per platform)

Description: Administrative (or Remote Help) access to a reproducing device on **each platform you manage** — Windows, iOS/iPadOS, and/or Android. **Why required:** The whole point of this product is to prove the change **applied on the device**, not just that Intune reported Succeeded. That requires reading the live saved network on the device: `netsh wlan show profiles` / effective proxy on Windows; the installed Wi-Fi payload on iOS/iPadOS; the saved network on Android. **How to verify:** On a Windows device you can run `WIFIDX-Validate-Device.ps1` locally; on Apple you can open **Settings** → **General** → **VPN & Device Management** to see the installed profile; on Android you can open the saved-network details. **How to implement if missing:** Arrange local admin, Intune **Remote Help**, or Intune → device → **Collect diagnostics** for Windows to package the MDM logs for offline reading.

2.5 Certificate dependency present (enterprise/802.1X profiles only)

Description: For an **Enterprise** (EAP / 802.1X) Wi-Fi profile, the **Trusted Root** and **SCEP/PKCS** certificate profiles the Wi-Fi profile references must already be deployed and healthy on the device. **Why required:** If the certificate the Wi-Fi profile expects is missing, the device **skips the Wi-Fi profile as pending certificates** — which looks exactly like "the change isn't applying." This toolkit checks for that dependency but **defers certificate deployment** itself to the certificate-Wi-Fi product. **How to verify:** On Android, the Company Portal `Omadmlog` shows *"Skipping Wifi profile … because it is pending certificates"* when the cert is absent. On Windows, confirm the Trusted Root and SCEP/PKCS profiles show **Succeeded** on the device before blaming the Wi-Fi profile. **How to implement if missing:** Deploy/repair the certificate chain using the certificate-Wi-Fi solution, then return here. Do not proceed with Wi-Fi remediation until the certificates are healthy.

2.6 A reproducing device and a known-good device

Description: One device that shows the stale/old Wi-Fi setting **and** one comparable device where the change did apply (or a freshly-enrolled device that never had the old network). **Why required:** Comparing a device that already knew the SSID against one that did not is the fastest way to confirm the in-place-update crux (Branch A). **How to verify:** Intune → the Wi-Fi profile → **Device status** lists both; on the devices, the saved-network details differ. **How to implement if missing:** Enroll or reset one test device so it has **never** joined the SSID, and keep one device that already had the old network.

2.7 Clarity on whether Cisco Meraki is in scope

Description: Know whether the environment uses the **Meraki dashboard** (wireless infrastructure) and/or **Meraki Systems Manager** (Meraki's MDM), and have read access to whichever is present. **Why required:** If Meraki SM is co-managing the device, or the Meraki dashboard enforces the setting at the

SSID, Intune is not authoritative and no Intune change will reflect (Branch D). **How to verify:** Ask whether devices were ever enrolled in Meraki SM; on an Apple device, **Settings** → **General** → **VPN & Device Management** shows **which MDM** owns the device. In the Meraki dashboard, **Wireless** → **Configure** shows SSID/group-policy settings. **How to implement if missing:** Obtain read access to the Meraki dashboard and/or SM console for the affected site. Where a Meraki step is version-specific, **verify against current Cisco Meraki documentation.**

Checklist

- Intune Administrator / Policy & Profile Manager role — Entra Roles
- Graph read scopes consented — `WIFIDX-Preflight.ps1` signs in
- Graph write scope available for the remediation helper (optional)
- Access to one affected device **per platform** you manage
- Certificate dependency confirmed healthy (enterprise/802.1X profiles only)
- One affected device **and** one known-good / never-joined device identified
- Meraki footprint understood (dashboard vs Systems Manager) and read access obtained
- Naming convention adopted (corrected profiles + pilot group named exactly as above)

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Diagnostic Playbook — Wi-Fi Changes Not Applying

This is the core of the solution: a **symptom-driven tree** for an Intune **Wi-Fi configuration profile** that will not update or reflect a change. Start from what you observe, follow the branch, run the exact checks, and land on the one cause that owns the failure. Then apply the fix and re-run the validation gates in `05-testing.md`.

Golden rule: a Wi-Fi change travels **Assignment** → **Sync/delivery** → **(certificate dependency)** → **application to the saved network** → **report**. Intune's **Succeeded** proves *delivery*, not that the live saved network on the device was rewritten. The **first** layer that fails is the root cause. For a *changed* profile, application-to-the-saved-network (Branch A) is where it silently stalls most often — check it before you touch anything else.

> ***Deferral, so you don't chase the wrong thing.*** *General "assigned but not applying" diagnosis (group/filter scope, sync health, generic CSP errors, policy conflicts) is owned by the* ***Intune Device Configuration & Policy Troubleshooting Toolkit*** *— this playbook assumes assignment and sync are healthy and focuses on the* ***Wi-Fi-specific*** *causes. First-time 802.1X* ***certificate*** *deployment is owned by the certificate-Wi-Fi product. If Step 0 shows the profile isn't even assigned or the device isn't syncing, fix that there first.*

Step 0 — Establish the facts (always do this first)

Do not change anything until you can answer all six. `WIFIDX-Preflight.ps1` and `WIFIDX-Get-WifiProfiles.ps1` collect most of this for you.

1. **Which platform?** Windows, iOS/iPadOS, or Android — the saved-network behavior and the proxy fields differ per platform (Branch F). Diagnose one platform at a time.

1. **Which profile, and what exactly changed?** Identify the exact Intune Wi-Fi profile and the field you edited — **PSK/password**, **proxy**, **SSID**, **security type**, or **auto-connect**. The change type points to the branch: any in-place edit → Branch A; a proxy edit → also Branch B.

1. **Is it an in-place edit of an SSID the device already knows?** Confirm whether the affected devices had **already joined** this SSID before the change. If yes, you are almost certainly in **Branch A** — Intune does not reliably overwrite a known network.

1. **What is the reported state vs the applied state?** In Intune, open the profile → **Device status **and note** Succeeded / Error / Not applicable **for the affected device. Then read the** actual **live setting on the device (Windows: `WIFIDX-Validate-Device.ps1`; Apple/Android: the saved-network details).** Succeeded + wrong-on-device = Branch A or E**, not a delivery failure.

1. **Are there duplicate or conflicting Wi-Fi profiles?** Run `WIFIDX-Get-WifiProfiles.ps1` — more than one Intune Wi-Fi profile targeting the same devices for the **same SSID** is Branch C.

1. **Is Cisco Meraki in play?** Was the device ever enrolled in **Meraki Systems Manager**? Does the **Meraki dashboard** enforce anything at the SSID (proxy, splash, auth)? If either is true, Intune may not be authoritative — Branch D.

Now pick the branch that matches your symptom.

Branch A — Modified profile does not update the existing SSID in place

Symptom: You changed a field (password, proxy, security type, auto-connect) on an existing Wi-Fi profile and saved it. Intune shows **Succeeded**, but devices that had **already joined** the SSID keep the **old** settings. New / never-joined devices may pick up the change fine — which is the tell.

Owner layer: Application to the saved network. **This is the crux of this product.**

Why it happens: A device that has joined an SSID keeps it as a **known / saved network**. When Intune re-delivers the edited payload, the device does **not** reliably overwrite the network it already has. Microsoft documents the canonical case (a changed password): after you change the password on an existing profile, *"some users may not get the new profile."* The same is observed for proxy, security-type, and auto-connect edits. The payload is delivered (hence **Succeeded**) but the live saved network is not rewritten.

How to confirm:

1. **Compare a joined device to a never-joined device.** On the reproducing device (already knew

`CORP-WIFI`), read the live setting:

• **Windows:** `WIFIDX-Validate-Device.ps1` (wraps `netsh wlan show profile name="CORP-WIFI" key=clear` and the effective proxy). The old value is still there.

• **iOS/iPadOS: Settings** → **General** → **VPN & Device Management** → the management profile → confirm the Wi-Fi payload; and **Settings** → **Wi-Fi** → **(i) next to CORP-WIFI** for the live proxy.

• **Android:** long-press the saved **CORP-WIFI** network → view details / proxy.

On a freshly-enrolled device that **never** joined `CORP-WIFI`, the **new** value is present. If new devices are correct and only previously-joined devices are wrong, that difference **is** the confirmation.

1. **Confirm delivery actually happened.** Intune → profile → **Device status** = Succeeded, and `WIFIDX-Report-ProfileState.ps1` flags the device as **reported-Succeeded but applied-mismatch**.

Fix — deploy a corrected replacement profile, do not edit in place:

1. **Create a NEW Wi-Fi profile** with the corrected settings rather than editing the old one:

• **Windows: Devices** → **Configuration** → **Create** → **New policy** → Platform **Windows 10 and later → **Profile type** Templates → Wi-Fi **(or** Settings catalog → Wi-Fi**)** → Create**. Name it `WIFIDX - WiFi - Corrected - Windows - Pilot`. Enter SSID `CORP-WIFI` and the corrected settings.

• **iOS/iPadOS: Devices** → **Configuration** → **Create** → Platform **iOS/iPadOS** → **Templates → Wi-Fi → Create**. Name it `WIFIDX - WiFi - Corrected - iOS - Pilot`.

• **Android: Devices** → **Configuration** → **Create** → the matching **Android Enterprise** profile type → **Wi-Fi** → **Create**. Name it `WIFIDX - WiFi - Corrected - Android - Pilot`.

1. **Assign it to** `WIFIDX - Pilot Devices` only (the Assigned pilot group from §2.6).

2. **Retire the old profile from the same devices.** To force the device to drop the stale saved network, **unassign the old profile from the pilot group** (or delete it once the replacement is proven). Removing the old Wi-Fi profile removes its saved network from enrolled devices, so the replacement installs cleanly instead of colliding with a known network.

1. **Where the SSID/security itself changed (not just a password):** the most reliable pattern is **remove-and-re-add** — remove the old saved network, then let the corrected profile create it fresh. `WIFIDX-Remediation.ps1 -WhatIf:$false` can remove the stale Windows saved network (`netsh wlan delete profile name="CORP-WIFI"`) as part of this, so the corrected profile re-adds it on the next sync. On Apple/Android, unassign/reassign the profile achieves the equivalent.

1. **If you must keep users online during the change:** deploy a **guest Wi-Fi** profile (`WIFIDX - WiFi - Guest - Fallback`) with auto-connect enabled to all users, as Microsoft recommends, so a device that briefly loses the corporate network can still reach the internet while the corrected profile lands.

Re-validate: run the Branch A gate in `05-testing.md` — the live saved network on a previously-joined device must now show the **new** value, not just Succeeded in Intune.

Branch B — Proxy set wrong, in the wrong field, or as URL-vs-hostname

Symptom: The device connects to `CORP-WIFI` but **all traffic breaks** (no browsing, auth failures), or the proxy simply never takes effect. Often paired with Branch A after a proxy edit.

Owner layer: The proxy sub-settings inside the Wi-Fi payload.

Why it happens: A Wi-Fi profile offers **two mutually-exclusive** proxy modes, and the fields mean different things. Putting a value in the wrong shape is the single most common proxy mistake.

The rule — memorize this:

- **Manual proxy = a host + a separate port.** It is a **hostname or IP address**, **never a URL** —

no `http://`, no `/path`. A slash or scheme in this field is sent verbatim and fails.

- **Automatic proxy = a PAC URL.** It is a **full URL** to a `.pac` script, **with** scheme —

e.g. `http://wpad.corp.example/proxy.pac`. A bare hostname here is wrong.

Exact fields per platform (verified against Microsoft Learn):

- **Windows** — profile → **Company proxy settings**:
- **None**
- **Manually configure** → **Proxy server IP address** + **Port number** (host/IP + port).
- **Automatically configure** → the **URL** pointing to a PAC script,

e.g. `http://wpad.corp.example/proxy.pac`.

- **iOS/iPadOS & macOS** — profile → **Proxy settings**:
- **None**
- **Manual** → **Proxy server address** (as an IP/host) + **Port number**.
- **Automatic** → **Proxy server URL** that contains the configuration file, e.g.

`http://wpad.corp.example/proxy.pac`.

- **Android Enterprise** — proxy support depends on the profile type; where a manual proxy is offered

it is a **host + port**, and automatic is a **PAC URL**. Some Android profile types do not expose a Wi-Fi proxy at all — **verify against current Microsoft documentation** for the specific Android enrollment type before assuming the field exists.

How to confirm:

1. **Read what you actually entered.** `WIFIDX-Get-WifiProfiles.ps1` exports each profile's proxy

mode and value. Look for a manual field containing `http://` or a `/` (wrong), or an automatic field with a bare host and no `.pac` (wrong).

1. **Read the effective proxy on the device.** Windows: `WIFIDX-Validate-Device.ps1` reports the

effective connection proxy. Apple: **Settings** → **Wi-Fi** → **(i)** → **HTTP Proxy**. Confirm it matches intent.

1. **If it's a PAC:** confirm the device can actually **reach** the PAC URL host (it must resolve and

serve the `.pac`). A PAC URL the device can't reach = no proxy applied.

Fix: Set the **correct mode** and put the value in the **correct shape** — manual = host + port (no scheme/path); automatic = full PAC URL. Because this is an edit to an existing profile, apply it via the **corrected replacement profile** from Branch A (not an in-place edit), then re-validate the effective proxy on the device.

Branch C — Duplicate or conflicting Wi-Fi profiles

Symptom: Settings flip between values, the profile intermittently shows **failing**, or your change is overwritten by "another" profile. On **Android Enterprise** fully managed / dedicated / corporate-owned work profile devices, **all** Wi-Fi profiles may report **failing** at once.

Owner layer: Multiple Wi-Fi payloads competing for the same SSID.

Why it happens: Two (or more) Intune Wi-Fi profiles targeting the same devices for the **same SSID** each try to own the saved network; the effective value becomes last-writer/undefined. And per Microsoft: for Android Enterprise fully managed/dedicated/COBO, **when you deploy more than one Wi-Fi profile and one fails, they all report as failing** — even the ones still working — which masks which profile is really the problem.

How to confirm:

1. **List every Wi-Fi profile and its targets.** `WIFIDX-Get-WifiProfiles.ps1` groups profiles by

SSID and flags any SSID targeted by more than one profile at the same devices. That list is your duplicate/conflict set.

1. **Check for a leftover old profile.** The pre-change profile you thought you replaced may still be

assigned to the same group — it re-asserts the old value every sync.

Fix: Establish **one profile per SSID per device**. Keep the `WIFIDX - WiFi - Corrected - <platform> - Pilot` profile and **unassign/delete** every other profile targeting `CORP-WIFI` on those devices. On Android, reduce to a single Wi-Fi profile per device so a single failure stops poisoning the report. Re-validate the aggregate report is clean.

Branch D — Cisco Meraki is overriding the client settings

Symptom: Nothing you do in Intune reflects on the device, and the environment uses Cisco Meraki. The setting (proxy, auth, SSID behavior) appears to come from "somewhere else."

Owner layer: A Meraki system — **not** Intune — is authoritative for the setting.

Why it happens — two different Meraki products, commonly confused:

- **Meraki dashboard (wireless infrastructure).** APs, broadcast SSIDs, RADIUS, VLANs, and per-SSID

group policies. This is the **network side**. If the proxy/auth/splash is enforced here at the SSID, it is not a device saved-network setting at all, and **no Intune Wi-Fi profile edit will ever change it** — the dashboard owns it.

- **Meraki Systems Manager (SM) — Meraki's own MDM/UEM.** SM pushes its **own Wi-Fi Settings

payload **(SSID, auto-join, and —** proxying is only supported on Apple devices **— a proxy that is** None / Manual (hostname or IP + port) / Automatic (a URL, WPAD if blank)**). If a device is enrolled in SM, it can carry a** second managed Wi-Fi payload for the same SSID**, masking your Intune change.

How to tell which system is authoritative:

1. **On Apple, check which MDM owns the device. Settings → General → VPN & Device Management →**

read the management profile's **name/organization**. An Apple device can be **MDM-enrolled to only one server** — if it says **Meraki**, it is **not** taking Intune Wi-Fi profiles at all (and vice versa). This alone resolves many "Intune isn't applying" cases.

1. **Look for a second Wi-Fi payload.** On the device, is there a Meraki-authored Wi-Fi profile for

`CORP-WIFI` alongside (or instead of) Intune's? On Windows, `WIFIDX-Validate-Device.ps1` shows the saved profile; a Meraki-managed one indicates SM co-management.

1. **Check the Meraki dashboard for SSID-level enforcement.** In the dashboard, **Wireless →

Configure → **the SSID → group policy / proxy / splash settings. If the behavior is set here, the dashboard is authoritative.** Verify against current Cisco Meraki documentation** — dashboard navigation changes over time.

Fix (decide authority first, then act):

- **If the setting is enforced at the Meraki dashboard SSID:** change it **in the Meraki dashboard**,

not in Intune. Intune should not also push a conflicting client-side value for that SSID.

- **If the device is enrolled in Meraki SM:** decide the **single** MDM of record. If Intune is to

own Wi-Fi, **unenroll from Meraki SM** (or remove SM's Wi-Fi payload) so the Intune profile is the only Wi-Fi authority; on Apple this may require re-enrollment because only one MDM can own the device. If Meraki SM is to own Wi-Fi, make the change **in SM** and stop pushing the Intune Wi-Fi profile to those devices.

- **Never leave two MDMs pushing the same SSID.** One system owns Wi-Fi; the other must not target

it. Flag any SM-side step as **verify against current Cisco Meraki documentation**.

Re-validate: confirm the device now has exactly one managed Wi-Fi payload for `CORP-WIFI`, from the authoritative system, showing the intended value.

Branch E — Profile reports Succeeded, device is unchanged (delivery vs application)

Symptom: Intune says **Succeeded** everywhere, but the live setting on the device is wrong — and you have already ruled out an obvious in-place-known-network case, or you want to confirm it's a reporting/application gap rather than a real failure.

Owner layer: The gap between **reported** state (delivery) and **applied** state (the live saved network).

Why it happens: Succeeded in Intune means the payload was **delivered and accepted** — not that the OS rewrote a network it already had (Branch A), and not that reporting is even current. Two specific reporting traps per Microsoft:

• **Android reporting error:** a Wi-Fi profile can **report as failing while actually working** — a known reporting bug fixed by updating the **Intune app to version 2021.05.02 or later**.

• **Android aggregate failure:** one failing Wi-Fi profile makes **all** report failing (see Branch C) — the report is lying about the working ones.

How to confirm:

1. **Trust the device, not the console.** `WIFIDX-Report-ProfileState.ps1` lists, per device, the **reported** state next to the **applied** state and flags mismatches. A row that is *Succeeded / applied-mismatch* is the delivery-vs-application gap.

1. **Read the platform log** to see the actual application result:

• **Windows:** Event Viewer → **Applications and Services Logs → Microsoft → Windows → DeviceManagement-Enterprise-Diagnostic-Provider → Admin**. Look for the WiFiConfigurationServiceProvider** node-set-value result.

• **Android:** Company Portal `Omadmlog` (read with CMTrace, filter `wifimgr`) — look for *"Applied profile"* vs *"Skipping Wifi profile … because it is pending certificates."*

• **iOS/iPadOS:** the device Console log — *"Profile … installed."*

1. **Check the Intune app version on Android** if the profile "fails but works."

Fix: If it's the known Android reporting bug, **update the Intune app to 2021.05.02+**. If the log shows the payload was accepted but the live network is stale, it's the Branch A application gap → apply the corrected replacement profile. If the log shows *pending certificates*, it's the dependency in §2.5 → repair certificates first. Then re-validate applied state on the device.

Branch F — Platform-specific quirks (Windows vs iOS vs Android)

Symptom: The change reflects on one platform but not another, or the same steps behave differently across your fleet.

Owner layer: Per-platform Wi-Fi handling.

Key differences to check:

• **Windows.** Saved networks are visible with `netsh wlan show profiles`; a stale one can be removed with `netsh wlan delete profile name="CORP-WIFI"` (what `WIFIDX-Remediation.ps1` does under `-WhatIf:$false`). Applied state is logged under **DeviceManagement-Enterprise-Diagnostic-Provider → Admin**. Error `0x87d1fde8` appears when a Wi-Fi profile targets a device with **no Wi-Fi hardware** — scope the profile to Wi-Fi-capable devices with an assignment filter.

• **iOS/iPadOS.** Wi-Fi lives **inside the MDM profile**; a device is **MDM-enrolled to only one server **(critical for Branch D). Proxy =** None / Manual (address + port) / Automatic (Proxy server URL)**. To force a changed network to reinstall, unassign then reassign the corrected profile.

- **Android Enterprise.** Behavior differs by enrollment type (fully managed / dedicated / work profile); one failing Wi-Fi profile makes **all** report failing; the "fails but works" reporting bug needs Intune app **2021.05.02+**. Proxy support varies by profile type — **verify against current Microsoft documentation** for your Android enrollment type. Certificate dependency (§2.5) is the top cause of a skipped Wi-Fi profile on Android.

Fix: Apply the platform-appropriate remediation from the branch that matches the symptom, using the corrected replacement-profile pattern, and validate on a device **of that platform** — never assume a fix on one platform proves the others.

After any fix — re-validate that it applied on a device

Do not stop at "Intune shows Succeeded." Force a sync, allow a check-in cycle, then run `WIFIDX-Validate-Device.ps1` (Windows) or read the live saved-network setting (Apple/Android) and confirm the **new** value is actually present on a **previously-joined** device. Use the gates in `05-testing.md`. Only promote the corrected profile from `- Pilot` to `- Prod` once a previously-joined device passes.

5. Testing & Validation

The single rule of this product: **Intune saying "Succeeded" is not a pass.** A pass is the **new setting present on a device that previously joined the SSID**. Every gate below is written that way — it validates *applied* state, not *reported* state. Run them on a `WIFIDX - Pilot Devices` member that already knew `CORP-WIFI`, and only promote to `- Prod` after every gate passes.

Each gate: **What you're proving · How to check · PASS / FAIL.**

Gate 0 — Reproduce and baseline (before any fix)

Proving: the problem is real and you have a device that shows it. **How to check:** On a previously-joined device, read the live setting — Windows: `WIFIDX-Validate-Device.ps1`; Apple: **Settings** → **Wi-Fi** → **(i) next to CORP-WIFI**; Android: saved network details. Compare to Intune → profile → **Device status**.

- **PASS (problem confirmed):** Intune shows **Succeeded** *and* the device still shows the **old** value. (This is the Branch A/E signature — proceed to fix.)
- **FAIL (different problem):** Intune shows **Error / Not applicable**, or the device isn't syncing → this is a delivery/assignment problem — go to the configuration toolkit first, not this playbook.

Gate 1 — Corrected replacement profile exists and is scoped (Branch A)

Proving: you built a **new** profile rather than editing the old one in place. **How to check:** Intune → **Devices** → **Configuration** → confirm `WIFIDX - WiFi - Corrected - <platform> - Pilot` exists, has the corrected settings, and is assigned to `WIFIDX - Pilot Devices` only. Confirm the **old** profile is unassigned from that group.

- **PASS:** the corrected profile exists and is the **only** Intune Wi-Fi profile targeting `CORP-WIFI` on the pilot devices.
- **FAIL:** you edited the original in place, or both old and new profiles still target the group (→ Branch C).

Gate 2 — The change is APPLIED on a previously-joined device (the core gate)

Proving: the device that already knew `CORP-WIFI` now has the **new** value — the exact failure this product exists to fix. **How to check:** Force a sync, allow one check-in cycle, then read the live saved network:

- **Windows:** `WIFIDX-Validate-Device.ps1` — the saved profile / effective proxy shows the new value.
- **iOS/iPadOS: Settings** → **Wi-Fi** → **(i)** shows the new proxy/settings; management profile lists the corrected payload.
- **Android:** saved-network details show the new value.
- **PASS:** the **new** value is present on a **previously-joined** device (not just a fresh one).
- **FAIL:** the device still shows the old value → the stale saved network was not replaced; run `WIFIDX-Remediation.ps1 -WhatIf:$false` to remove the stale saved network (or unassign/reassign on Apple/Android) and re-test.

Gate 3 — Proxy is in the correct shape and effective (Branch B)

Proving: the proxy is in the right mode and field, and traffic actually works. **How to check:** `WIFIDX-Get-WifiProfiles.ps1` shows the profile's proxy mode/value; confirm:

- **Manual** = host/IP + port, with **no** `http://` and **no** `/path`.
- **Automatic** = a full **PAC URL** (scheme + `.pac`) the device can reach.

Then on the device, confirm the **effective** proxy (Windows: `WIFIDX-Validate-Device.ps1`; Apple: **Wi-Fi** → **(i)** → **HTTP Proxy**) and that browsing/authentication succeeds over `CORP-WIFI`.

- **PASS:** proxy value is correctly shaped, effective on the device, and traffic works.
- **FAIL:** a URL/slash sits in a manual field, a bare host sits in an automatic field, or the PAC URL is unreachable → correct the shape in the replacement profile and re-test.

Gate 4 — Exactly one Wi-Fi authority for the SSID (Branches C & D)

Proving: only one system/profile owns `CORP-WIFI` on the device. **How to check:** `WIFIDX-Get-WifiProfiles.ps1` confirms a single Intune profile per SSID per device. If Meraki is present: on Apple, **Settings** → **General** → **VPN & Device Management** shows a single MDM of record;

confirm no second (Meraki SM) Wi-Fi payload for `CORP-WIFI`, and that any dashboard-enforced setting is intended.

- **PASS:** one Intune Wi-Fi profile owns the SSID; no competing Intune profile and no competing Meraki SM payload; dashboard enforcement (if any) is deliberate.
- **FAIL:** duplicate Intune profiles, or a Meraki SM payload also targets the SSID → resolve to a single authority (Branch C/D).

Gate 5 — Reported vs applied reconciled (Branch E)

Proving: the console now matches reality. **How to check:** `WIFIDX-Report-ProfileState.ps1` — the previously-mismatched device now reads **Succeeded / applied-match**. On Android where a profile "failed but worked," confirm the Intune app is **2021.05.02 or later**.

- **PASS:** reported and applied state agree for the pilot devices.
- **FAIL:** still Succeeded/applied-mismatch → return to Gate 2 (application gap) or Branch D (wrong authority).

Gate 6 — New-device regression

Proving: the fix also works for devices that never had the old network. **How to check:** On a freshly-enrolled device that **never** joined `CORP-WIFI`, confirm the corrected profile installs the network with the new value on first sync.

- **PASS:** new device gets the correct network first time.
- **FAIL:** new device also wrong → the profile itself (not the stale-network problem) is misconfigured → re-check Branch B and the profile settings.

Promotion gate — Pilot → Production

Only after Gates 0–6 pass on the pilot group:

1. Rename/duplicate the corrected profile to `WIFIDX - WiFi - Corrected - <platform> - Prod` and assign it to the production group.
1. Remove every superseded/old Wi-Fi profile for `CORP-WIFI` from production.
2. Re-run `WIFIDX-Report-ProfileState.ps1` against production and confirm applied-state matches for a sample of **previously-joined** production devices before declaring done.

6. Troubleshooting

This guide covers getting the **method, scripts, and access** working. To diagnose an actual Wi-Fi change that isn't applying, use the **Diagnostic Playbook** (`03-implementation.md`) — that is the symptom tree. This file is about the tooling and the process itself.

Connecting to Microsoft Graph

`Connect-MgGraph` **fails or the module is missing.**

- Install the SDK: `Install-Module Microsoft.Graph -Scope CurrentUser`.
- Confirm it is present: `Get-Module Microsoft.Graph -ListAvailable`.
- Run PowerShell as your admin account and re-run; consent to the requested scopes at the prompt.

Wrong cloud / "no profiles found" in a Government tenant.

- Pass the matching environment: `-Environment USGov` (GCC High) or `-Environment USGovDoD` (DoD).

The SDK then targets `graph.microsoft.us` instead of `graph.microsoft.com`. The **admin portal** for US Gov is `intune.microsoft.us`. If a script returns nothing, first confirm you connected to the correct environment and tenant.

"Insufficient privileges" or consent required.

- Read-only scripts need `DeviceManagementConfiguration.Read.All`,

`DeviceManagementManagedDevices.Read.All`, and `Group.Read.All`. The remediation helper's group-create needs `Group.ReadWrite.All`.

- If your tenant blocks user consent, have a Global Administrator grant admin consent for the

Microsoft Graph Command Line Tools app once (Entra → **Enterprise applications**).

Scripts return no Wi-Fi profiles even though they exist.

- Confirm your account can read configuration profiles and that you signed in to the correct tenant

and environment. Settings-catalog Wi-Fi policies are reported separately from legacy Wi-Fi templates — open a settings-catalog policy in the console to read its SSID if the export shows "(settings-catalog — open policy for SSID)".

Reading applied state on the device

`WIFIDX-Validate-Device.ps1` **shows no saved profiles.**

- The device may have no WLAN adapter (Windows error `0x87d1fde8` also indicates a Wi-Fi profile

pushed to non-Wi-Fi hardware), or it genuinely never joined the SSID. A never-joined device is your **control** for Branch A — it should show the new settings once the corrected profile applies.

I can't get onto the affected device.

- Windows: Intune → **Devices** → **the device** → **Collect diagnostics** packages the MDM logs (including

DeviceManagement-Enterprise-Diagnostic-Provider) for offline reading.

• Apple/Android: use Intune **Remote Help** or have the user read **Settings** → **Wi-Fi** → **(i)** /

saved-network details to you.

Windows Event Viewer has no Wi-Fi entries.

• Enable **Show Analytic and Debug Logs** (View menu), then look under **Applications and Services

Logs → Microsoft → Windows → DeviceManagement-Enterprise-Diagnostic-Provider → Admin **for the** WiFiConfigurationServiceProvider** node-set-value result.

Interpreting states

Intune says Succeeded but the device is unchanged.

• That is the core symptom, not a tooling bug. **Succeeded = delivered**, not applied. Go to Branch A

(in-place edit of a known network) or Branch E (delivery-vs-application), and prove applied state on a **previously-joined** device.

Android: the profile shows Failed but Wi-Fi works.

• Known reporting bug — update the **Intune app to 2021.05.02 or later**. Also, on Android Enterprise

fully managed/dedicated/COBO, **one** failing Wi-Fi profile makes **all** report failing (Branch C) — reduce to one Wi-Fi profile per device.

Android: "Skipping Wifi profile … because it is pending certificates."

• The enterprise/802.1X Wi-Fi profile depends on a Trusted Root + SCEP/PKCS certificate that isn't on

the device yet. Fix the certificate chain first (certificate-Wi-Fi product), then the Wi-Fi profile.

Remediation helper

The helper made no change.

• By design: `WIFIDX-Remediation.ps1` defaults to `-WhatIf` and only previews. Re-run with

`-WhatIf:$false` after reviewing the planned action.

`-RemoveStaleOnDevice` **deleted the network and the device is now offline.**

• Expected transient state: the corrected Intune profile re-adds the network on the next sync. Trigger

a sync (**Settings** → **Accounts** → **Access work or school** → **Info** → **Sync**). Keep a **guest Wi-Fi** fallback profile deployed so devices stay online during the swap.

• **Always confirm the corrected replacement profile exists and is assigned before removing the stale

saved network** — otherwise there is nothing to re-add it.

The helper won't create the pilot group.

• Group creation needs `Group.ReadWrite.All`. Have a Global Administrator consent, or create

`WIFIDX - Pilot Devices` (Security, **Assigned**) by hand in Entra.

When the method stalls

• **You "fixed" it in place and it still won't update.** Editing the live profile is the trap. Build a **new** corrected profile, scope it to the pilot group, retire the old one, and re-test on a previously-joined device.

• **It applies on new devices but never on old ones.** Textbook Branch A — the stale saved network is not being overwritten. Remove-and-re-add (or unassign/reassign) is the fix.

• **Nothing in Intune ever reflects and Meraki is present.** Determine authority (Branch D): on Apple a device has exactly **one** MDM of record; the Meraki **dashboard** may own the SSID at the infrastructure level. Change it where it's actually owned. **Verify Meraki-side steps against current Cisco Meraki documentation.**

• **Government tenant specifics.** Behavior matches Commercial; only endpoints differ. Where a GCC High detail is uncertain, **verify for GCC High** rather than assume parity.

7. Best Practices

Practices that prevent "Wi-Fi change not applying" from recurring, drawn from the crux behaviors this toolkit diagnoses.

Treat a Wi-Fi change as a new profile, not an edit

• **Never edit a live Wi-Fi profile's SSID, security type, or PSK in place** and expect previously-joined devices to update — they often will not. Build a **new** profile (`WIFIDX - WiFi - Corrected - <platform> - Pilot`), pilot it, then retire the old one.

• Editing a **non-network** attribute (e.g. auto-connect) in place is lower-risk, but still validate applied state on a previously-joined device before trusting it.

One profile — and one authority — per SSID per device

• Keep exactly **one** Intune Wi-Fi profile per SSID targeting a given device. Duplicates produce last-writer/undefined values and, on Android, make **all** Wi-Fi profiles report failing.

• If Cisco Meraki is present, decide **one** system of record for Wi-Fi. Let the **Meraki dashboard** own SSID/infrastructure enforcement, or let an MDM (Intune **or** Meraki Systems Manager, not both) own the client Wi-Fi payload — never two MDMs pushing the same SSID. On Apple, remember only **one** MDM can own the device.

Get the proxy shape right, every time

• **Manual proxy = host/IP + port.** No scheme, no path — a hostname, not a URL.

- **Automatic proxy = a PAC URL** (`http://host/….pac`) the device can actually reach.
- Document which mode your network uses so admins stop pasting URLs into the manual field.

Name objects so they audit and clean up cleanly

- Use the convention `WIFIDX - WiFi - Corrected - <platform> - <stage>` and the **`WIFIDX - Pilot

Devices`** Assigned group. Reuse the exact names in scripts, testing, and troubleshooting so the old profile is obvious when it's time to retire it.

Scope Wi-Fi profiles to devices that can use them

- Target Wi-Fi profiles with an **assignment filter for Wi-Fi-capable hardware** to avoid the Windows `0x87d1fde8` error on devices with no Wi-Fi adapter (which otherwise pollutes your failure report).
- Match the profile platform and Android enrollment type to the device — proxy and payload support vary by type.

Keep a fallback so a change never strands users

- Deploy a **guest Wi-Fi** profile with auto-connect to all users, per Microsoft's guidance, so a device that briefly drops the corporate network during a change can still reach the internet and re-sync.

Validate on the device, not in the console

- Make **applied-state on a previously-joined device** the definition of done. Use `WIFIDX-Report-ProfileState.ps1` to catch **Succeeded / applied-mismatch** rows before users report them.
- Keep Android's **Intune app updated (2021.05.02+)** so the "fails but works" reporting bug doesn't send you chasing phantom failures.

Keep vendor-specific and certificate steps in their lane

- For 802.1X **certificate** health, use the certificate-Wi-Fi product; a missing cert makes a Wi-Fi profile skip as *pending certificates*, which mimics "not applying."
- For **Meraki** console internals, **verify against current Cisco Meraki documentation** — dashboards and SM change over time.

Government / sovereign-cloud note

- In **GCC / GCC High / DoD**, the Wi-Fi profile behavior and the remediation pattern are the same; only the portals/endpoints differ (`intune.microsoft.us`, Graph `graph.microsoft.us`, `Connect-MgGraph -Environment USGov/USGovDoD`). The **FIPS** toggle on a Windows Wi-Fi profile is

the one setting more likely to matter for federal buyers. Where a GCC High specific is uncertain, **verify for GCC High** rather than assume parity.

8. Deliverables Summary

Everything in this package, and how it fits together. The documents are the method; the scripts turn the method into read-only evidence and a guarded fix.

Documents

- **Executive Summary** — why a "Succeeded" Wi-Fi profile leaves the device unchanged, and what this solution does about it.
- **1. Solution Overview** — how a Wi-Fi change travels to the saved network, why an in-place edit stalls, and how Meraki fits.
- **2. Requirements Guide** — roles, Graph access, per-platform device access, the certificate dependency, and the Meraki footprint — each with verify / implement-if-missing. Plus the naming convention used everywhere.
- **3. Diagnostic Playbook** — Step 0 (establish the facts) and labelled branches:
- **A** — modified profile not updating the existing SSID in place (**the crux**).
- **B** — proxy wrong / wrong field / hostname-vs-URL.
- **C** — duplicate or conflicting Wi-Fi profiles.
- **D** — Cisco Meraki dashboard or Systems Manager overriding the client settings.
- **E** — reports Succeeded but device unchanged (delivery vs application).
- **F** — platform-specific quirks (Windows / iOS / Android).
- **5. Testing & Validation** — pass/fail gates that prove the change **applied on a device**, not just that Intune reported it.
- **6. Troubleshooting** — getting the method, scripts, and access itself working.
- **7. Best Practices** — how to stop it recurring.

PowerShell scripts (read-only by default; cloud-aware)

All Graph scripts accept `-Environment Global|USGov|USGovDoD` and connect with the matching sovereign endpoint. Read-only unless you explicitly run the remediation helper without `-WhatIf`.

- `WIFIDX-Preflight.ps1` — verify the Graph SDK, sign-in, scopes, and environment before you start.
- `WIFIDX-Get-WifiProfiles.ps1` — export every Intune Wi-Fi configuration profile, its assignments, and its proxy settings; group by SSID and **flag duplicates/conflicts**.
- `WIFIDX-Validate-Device.ps1` — on a Windows device, read the live saved network

(`netsh wlan show profiles`) and effective proxy — the applied-state ground truth.

• `WIFIDX-Report-ProfileState.ps1` — reconcile **reported** (Succeeded) vs **applied** state per device and flag Succeeded/applied-mismatch rows.

• `WIFIDX-Remediation.ps1` — guarded (`-WhatIf` default) helper to build the corrected/renamed replacement profile and/or remove the stale saved network so the corrected profile re-adds it.

Boundaries (what this does not cover — by design)

• **General config-profile diagnosis** (assignment, filters, sync, CSP conflicts) → the Intune Device Configuration & Policy Troubleshooting Toolkit.

• **802.1X certificate deployment** (SCEP/PKCS, trusted root, RADIUS) → the certificate-Wi-Fi and Wi-Fi deployment products.

• **Meraki console internals** → verify against current Cisco Meraki documentation.

Delivery

Delivered as a Word document and a PDF (cover, table of contents, all sections, and a scripts appendix), plus the five PowerShell scripts — bundled in a single downloadable package you keep and reuse across unlimited devices.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

WIFIDX-Get-WifiProfiles.ps1

Purpose: Enumerates every Intune Wi-Fi device-configuration profile (legacy Wi-Fi templates across Windows / iOS / Android, plus settings-catalog Wi-Fi policies), and for each reports the SSID, proxy mode/value, and the Included/Excluded Entra groups. It then groups profiles by SSID and FLAGS any SSID that is targeted by more than one profile (the duplicate/conflict set from Branch C). Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WIFIDX-Get-WifiProfiles.ps1 powershell -ExecutionPolicy Bypass -File .\WIFIDX-Get-WifiProfiles.ps1 -Environment USGov -Ssid CORP-WIFI Requires scopes DeviceManagementConfiguration.Read.All, Group.Read.All. EXPECTED RESULTS One block per Wi-Fi profile (name, platform, SSID, proxy mode + value, Included/Excluded groups), then a DUPLICATES/CONFLICTS section listing any SSID owned by 2+ profiles. Proxy values are checked for the common shape mistake (a URL/slash in a manual field, or a bare host in an automatic/PAC field) and flagged inline. .NOTES Intune Solutions — read-only. Cloud-aware (Global | USGov | USGovDoD). Detects duplicates/conflicts. > [CmdletBinding()] param([ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global', [string]$Ssid)

How to run: powershell -ExecutionPolicy Bypass -File .\WIFIDX-Get-WifiProfiles.ps1 powershell -ExecutionPolicy Bypass -File .\WIFIDX-Get-WifiProfiles.ps1 -Environment USGov -Ssid CORP-WIFI Requires scopes DeviceManagementConfiguration.Read.All, Group.Read.All. EXPECTED RESULTS One block per Wi-Fi profile (name, platform, SSID, proxy mode + value, Included/Excluded groups), then a DUPLICATES/CONFLICTS section listing any SSID owned by 2+ profiles. Proxy values are checked for the common shape mistake (a URL/slash in a manual field, or a bare host in an automatic/PAC field) and flagged inline. .NOTES Intune Solutions — read-only. Cloud-aware (Global | USGov | USGovDoD). Detects duplicates/conflicts. > [CmdletBinding()] param([ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global', [string]$Ssid)

WIFIDX-Preflight.ps1

Purpose: Confirms the workstation is ready to diagnose Intune Wi-Fi profiles: the Microsoft Graph SDK is installed, you can sign in to the correct cloud (Commercial / US Gov / US Gov DoD), the read scopes are consented, and at least one Wi-Fi configuration profile is visible. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WIFIDX-Preflight.ps1 powershell -ExecutionPolicy Bypass -File .\WIFIDX-Preflight.ps1 -Environment USGov Requires Microsoft Graph SDK and scope DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A short readiness report: SDK present, connected tenant + environment, scopes granted, and a count of Wi-Fi configuration profiles found. Any missing prerequisite is called out with the fix. .NOTES Intune Solutions — read-only. Cloud-aware (Global | USGov | USGovDoD). > [CmdletBinding()] param([ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\WIFIDX-Preflight.ps1 powershell -ExecutionPolicy Bypass -File .\WIFIDX-Preflight.ps1 -Environment USGov Requires Microsoft Graph SDK and scope DeviceManagementConfiguration.Read.All. EXPECTED RESULTS A short readiness report: SDK present, connected tenant + environment, scopes granted, and a count of Wi-Fi configuration profiles found. Any missing prerequisite is called out with the fix. .NOTES Intune Solutions — read-only. Cloud-aware (Global | USGov | USGovDoD). > [CmdletBinding()] param(

[ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

WIFIDX-Remediation.ps1

Purpose: Supports the reliable remediation from Branch A: rather than editing a Wi-Fi profile in place, you deploy a corrected REPLACEMENT profile and clear the stale saved network so it re-adds cleanly. This helper does two guarded things, each defaulting to -WhatIf (preview only): (a) -RemoveStaleOnDevice : on THIS Windows device, delete the stale saved network for -Ssid (netsh wlan delete profile) so the corrected Intune profile re-creates it on next sync. (b) -CreatePilotGroup : create the Assigned Entra group 'WIFIDX - Pilot Devices' (Graph) used to scope the corrected pilot profile. It does NOT edit or delete Intune profiles for you — creating the corrected profile is a deliberate console step (document 3) so you own its exact settings. Nothing changes until you re-run with -WhatIf:$false. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\WIFIDX-Remediation.ps1 -RemoveStaleOnDevice -Ssid CORP-WIFI Apply on device: powershell -ExecutionPolicy Bypass -File .\WIFIDX-Remediation.ps1 -RemoveStaleOnDevice -Ssid CORP-WIFI -WhatIf:$false Create pilot group: powershell -ExecutionPolicy Bypass -File .\WIFIDX-Remediation.ps1 -CreatePilotGroup -Environment USGov -WhatIf:$false On-device action needs elevation; -CreatePilotGroup needs Group.ReadWrite.All. EXPECTED RESULTS In -WhatIf: prints exactly what it WOULD do and changes nothing. Applied: removes the stale saved network (then advises a sync) and/or creates the Assigned pilot group, echoing the id. .NOTES Intune Solutions — GUARDED. Default -WhatIf. Cloud-aware (Global | USGov | USGovDoD). Confirm the corrected replacement profile exists BEFORE removing a stale saved network. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([switch]$RemoveStaleOnDevice, [string]$Ssid, [switch]$CreatePilotGroup, [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global', [switch]$WhatIf = $true) make -WhatIf the default while still honouring an explicit -WhatIf:$false if ($PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = [bool]$WhatIf } else { $WhatIfPreference = $true }

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\WIFIDX-Remediation.ps1 -RemoveStaleOnDevice -Ssid CORP-WIFI Apply on device: powershell -ExecutionPolicy Bypass -File .\WIFIDX-Remediation.ps1 -RemoveStaleOnDevice -Ssid CORP-WIFI -WhatIf:$false Create pilot group: powershell -ExecutionPolicy Bypass -File .\WIFIDX-Remediation.ps1 -CreatePilotGroup -Environment USGov -WhatIf:$false On-device action needs elevation; -CreatePilotGroup needs Group.ReadWrite.All. EXPECTED RESULTS In -WhatIf: prints exactly what it WOULD do and changes nothing. Applied: removes the stale saved network (then advises a sync) and/or creates the Assigned pilot group, echoing the id. .NOTES Intune Solutions — GUARDED. Default -WhatIf. Cloud-aware (Global | USGov | USGovDoD). Confirm the corrected replacement profile exists BEFORE removing a stale saved network. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([switch]$RemoveStaleOnDevice, [string]$Ssid, [switch]$CreatePilotGroup, [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global', [switch]$WhatIf = $true) make -WhatIf the default while still honouring an explicit -WhatIf:$false if ($PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = [bool]$WhatIf } else { $WhatIfPreference = $true }

WIFIDX-Report-ProfileState.ps1

Purpose: For a chosen Wi-Fi configuration profile, pulls the per-device REPORTED status from Intune (Succeeded / Error / Not applicable / Pending) so you can line it up against the APPLIED state you read on the device with WIFIDX-Validate-Device.ps1. The report highlights devices that Intune calls Succeeded —

those are exactly the ones to spot-check on the device for the Branch A stale-network / Branch E delivery-vs-application gap. Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WIFIDX-Report-ProfileState.ps1 -NameLike CORP-WIFI powershell -ExecutionPolicy Bypass -File .\WIFIDX-Report-ProfileState.ps1 -Environment USGov -NameLike "WIFIDX - WiFi - Corrected" Requires scopes DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS For each matching profile: a per-device table of reported status, and a summary count by status. A note lists the Succeeded devices to verify on-device (reported != applied is not visible to Intune — you must read the device). "No matching profiles" if the name filter matches nothing. .NOTES Intune Solutions — read-only. Cloud-aware (Global | USGov | USGovDoD). Reported vs applied reconciliation. > [CmdletBinding()] param([ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global', [string]$NameLike)

How to run: powershell -ExecutionPolicy Bypass -File .\WIFIDX-Report-ProfileState.ps1 -NameLike CORP-WIFI powershell -ExecutionPolicy Bypass -File .\WIFIDX-Report-ProfileState.ps1 -Environment USGov -NameLike "WIFIDX - WiFi - Corrected" Requires scopes DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS For each matching profile: a per-device table of reported status, and a summary count by status. A note lists the Succeeded devices to verify on-device (reported != applied is not visible to Intune — you must read the device). "No matching profiles" if the name filter matches nothing. .NOTES Intune Solutions — read-only. Cloud-aware (Global | USGov | USGovDoD). Reported vs applied reconciliation. > [CmdletBinding()] param([ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global', [string]$NameLike)

WIFIDX-Validate-Device.ps1

Purpose: The applied-state ground truth for Branch A/B/E. Run locally on an affected Windows device to read what the device ACTUALLY has for an SSID — the saved (known) network and its effective proxy — so you can compare it to what Intune reports. This is how you catch a profile that says "Succeeded" while the device is unchanged. Read-only (no changes to the device). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WIFIDX-Validate-Device.ps1 -Ssid CORP-WIFI Run in an elevated PowerShell ON the affected device. No Graph/cloud connection needed. EXPECTED RESULTS The saved profile details for the SSID (security/auth), whether it is a known network, and the effective WinHTTP/user proxy. Also lists ALL saved Wi-Fi profiles so a duplicate/stale one is visible. Prints "SSID not saved on this device" if the device never joined it. .NOTES Intune Solutions — read-only, on-device. No cloud connection. Compares APPLIED vs REPORTED state. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Ssid)

How to run: powershell -ExecutionPolicy Bypass -File .\WIFIDX-Validate-Device.ps1 -Ssid CORP-WIFI Run in an elevated PowerShell ON the affected device. No Graph/cloud connection needed. EXPECTED RESULTS The saved profile details for the SSID (security/auth), whether it is a known network, and the effective WinHTTP/user proxy. Also lists ALL saved Wi-Fi profiles so a duplicate/stale one is visible. Prints "SSID not saved on this device" if the device never joined it. .NOTES Intune Solutions — read-only, on-device. No cloud connection. Compares APPLIED vs REPORTED state. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Ssid)